From Gaussian AR(1) Belief Propagation to General Markov-Chain Diffusion Scores

Exact BP, continuous-message bottlenecks, and Gaussian-message approximation error

Prepared for the thesis project discussion

July 2026

Abstract

This note generalizes the Gaussian AR(1) belief-propagation derivation to arbitrary clean Markov-chain priors observed through coordinatewise Ornstein–Uhlenbeck noising. The main point is structural: Gaussianity is not needed for BP exactness. If the clean prior is a chain and the likelihood factorizes sitewise, then the posterior is again a chain, hence a tree factor graph, and sum-product BP computes exact posterior marginals. The diffusion score is then obtained from the exact identity

$$\nabla_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t \mathbb{E}[a_i \mid x]}{\Delta_t}.$$

Gaussianity only gives a finite-dimensional closure of the continuous BP messages. Outside the Gaussian case, messages are continuous probability functions and must be represented numerically or approximated. We therefore design two algorithms: reference grid BP and Gaussian projected BP. The grid is validated in the Gaussian AR(1) case against the exact precision-matrix score, and then used as a numerical proxy for the true score in a Laplace-innovation non-Gaussian chain. The experiments quantify the Gaussian-message approximation error at the posterior-mean and score levels.

Contents

1 Executive summary **3**

2 The diffusion setup **3**
2.1 Clean chain prior 3
2.2 Coordinatewise OU noising 4

3 Posterior factorization: the posterior remains a chain **4**

4 Generic sum-product BP and exactness on trees **5**
4.1 Generic factor graph notation 5
4.2 Why BP is exact on trees 6

5 Forward-backward BP for the posterior chain **6**

6 The score identity: every algebraic step **7**

7 What was special about the Gaussian AR(1) case? 9
7.1 Gaussian clean prior . . . 9
7.2 Gaussian message closure . . . 10
7.3 The key distinction . . . 11

8 The continuous non-Gaussian bottleneck 11

9 Approximation error and score error 11

10 Algorithm 1: reference grid BP 12
10.1 Grid representation . . . 12
10.2 Forward recursion . . . 12
10.3 Backward recursion . . . 13
10.4 Belief, posterior mean, and score . . . 13

11 Algorithm 2: Gaussian projected BP 13

12 Experimental models 14
12.1 Gaussian AR(1): validation model . . . 14
12.2 Laplace-innovation AR(1): non-Gaussian test model . . . 14

13 Error metrics 15

14 Numerical results from the report run 15
14.1 Gaussian validation of grid BP . . . 16
14.2 Laplace grid convergence . . . 17
14.3 Gaussian-message approximation error in the Laplace chain . . . 18
14.4 Example beliefs . . . 20

15 Code explanation 22
15.1 Configuration . . . 22
15.2 Densities and numerical normalization . . . 22
15.3 Transition matrices . . . 22
15.4 Grid BP implementation . . . 23
15.5 Gaussian projected BP implementation . . . 23
15.6 Exact Gaussian score functions . . . 23
15.7 Experiment runners . . . 24
15.8 How to run . . . 24

16 Interpretation and next steps 24

A Files in the delivered folder 25

B Source references 25

1 Executive summary

The Gaussian AR(1) calculation was useful, but it was not the most general statement. The correct general statement is:

$$\boxed{\text{Markov clean prior} + \text{coordinatewise OU noising} \Rightarrow \text{posterior chain}}$$

so

$$\boxed{\text{sum-product BP is exact, because chains are trees.}}$$

Then the diffusion score follows from the posterior mean:

$$\boxed{s_i(x,t) = \partial_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t m_i(x,t)}{\Delta_t}, \qquad m_i(x,t) = \mathbb{E}[a_i \mid x].}$$

The conceptual difference between the Gaussian and non-Gaussian cases is not exactness. The difference is representation:

Case	BP exact?	Message representation
Gaussian AR(1)	yes	two scalars: precision and information
Finite-state Markov chain	yes	finite vector
Continuous non-Gaussian Markov chain	yes	full function/density

Therefore the computational problem is:

$$\boxed{\text{In continuous non-Gaussian chains, exact BP propagates functions.}}$$

The Gaussian-message approximation forces those functions to remain Gaussian. Jerome's question is then naturally formalized as:

$$\boxed{\text{How large is the error caused by Gaussian message closure?}}$$

The code implements the following experimental pipeline:

1. Validate grid BP in the Gaussian AR(1) case, where the exact score is known by linear algebra.
2. In a Laplace-innovation AR(1) chain, estimate grid discretization error by comparing a coarse grid with a fine grid.
3. In the same Laplace chain, compare Gaussian projected BP with reference grid BP.
4. Convert all posterior-mean errors into score errors using the exact OU identity.

2 The diffusion setup

2.1 Clean chain prior

Let

$$a_{1:n} = (a_1, \ldots, a_n)$$

be clean latent variables. For now take each

$$a_i \in \mathcal{A} \subseteq \mathbb{R},$$

but the same factorization logic works for finite state spaces, vector-valued states, or mixed spaces with the appropriate sums/integrals.

The clean prior is assumed to be Markov along the sequence index:

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1}), \tag{1}$$

where μ is the initial density and K_i is a transition density or kernel.

This is the structural assumption. It does not assume Gaussianity. For example, K_i can be Gaussian, Laplace, mixture Gaussian, nonlinear, bounded-support, or discrete.

2.2 Coordinatewise OU noising

At diffusion time $t > 0$, we observe

$$x_i = \alpha_t a_i + \sqrt{\Delta_t}\, z_i, \qquad z_i \sim \mathcal{N}(0, 1), \tag{2}$$

independently over i, with

$$\alpha_t = e^{-t}, \qquad \Delta_t = 1 - e^{-2t} = 1 - \alpha_t^2. \tag{3}$$

The one-site likelihood is therefore

$$\ell_i(a_i; x_i) := p_t(x_i \mid a_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \tag{4}$$

Written explicitly,

$$\ell_i(a_i; x_i) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp\left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t}\right]. \tag{5}$$

Because the noises z_i are independent, the full likelihood factorizes:

$$p_t(x_{1:n} \mid a_{1:n}) = \prod_{i=1}^{n} \ell_i(a_i; x_i). \tag{6}$$

3 Posterior factorization: the posterior remains a chain

Proposition 1 (Posterior chain factorization)**.** *Assume the clean prior has the Markov factorization* (1) *and the likelihood factorizes as in* (6)*. Then the posterior is*

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{1}{Z_t(x)} \mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1}) \prod_{i=1}^{n} \ell_i(a_i; x_i), \tag{7}$$

where

$$Z_t(x) = p_t(x_{1:n})$$

is the evidence. Hence the posterior factor graph is a chain with unary observation factors.

Proof. Start from Bayes' rule:

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{p_0(a_{1:n}) p_t(x_{1:n} \mid a_{1:n})}{p_t(x_{1:n})}. \tag{8}$$

Now substitute the Markov prior:

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1}), \tag{9}$$

and the factorized likelihood:

$$p_t(x_{1:n} \mid a_{1:n}) = \prod_{i=1}^{n} \ell_i(a_i; x_i). \tag{10}$$

Therefore

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{1}{p_t(x_{1:n})} \left[\mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1}) \right] \left[\prod_{i=1}^{n} \ell_i(a_i; x_i) \right] \tag{11}$$

$$= \frac{1}{Z_t(x)} \mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1}) \prod_{i=1}^{n} \ell_i(a_i; x_i). \tag{12}$$

Every transition factor touches only adjacent variables (a_{i-1}, a_i), and every likelihood factor touches only one variable a_i. Thus no loops are created. The posterior graph is still a chain. □

Graphically, the posterior structure is

$$a_1 - a_2 - \cdots - a_n,$$

with one unary likelihood ℓ_i attached to each a_i. This is why the BP exactness argument applies.

4 Generic sum-product BP and exactness on trees

4.1 Generic factor graph notation

Let a distribution factorize as

$$p(a) = \frac{1}{Z} \prod_{f \in F} \psi_f(a_{\partial f}), \tag{13}$$

where F is the set of factors and ∂f is the set of variables adjacent to factor f. Let ∂i be the set of factors adjacent to variable a_i.

Sum-product BP passes two types of messages.

The variable-to-factor message is

$$m_{i \to f}(a_i) \propto \prod_{g \in \partial i \setminus f} m_{g \to i}(a_i). \tag{14}$$

The factor-to-variable message is

$$m_{f \to i}(a_i) \propto \int \psi_f(a_{\partial f}) \prod_{j \in \partial f \setminus i} m_{j \to f}(a_j) \prod_{j \in \partial f \setminus i} \mathrm{d}a_j. \tag{15}$$

For discrete variables, the integrals are replaced by sums.

Once messages have converged, or after the two sweeps on a tree, the belief at variable i is

$$b_i(a_i) = \frac{\prod_{f \in \partial i} m_{f \to i}(a_i)}{\int \prod_{f \in \partial i} m_{f \to i}(u) \, \mathrm{d}u}. \tag{16}$$

4.2 Why BP is exact on trees

On a tree, deleting one edge separates the graph into two disconnected components. Therefore a message sent across an edge can be interpreted exactly as the partial marginal contribution of the whole subtree behind that edge.

For example, suppose a message is sent from a factor-side subtree to a boundary variable a_i. The factor-to-variable update integrates all variables inside that subtree except the boundary variable. Since no loop reconnects that subtree to the rest of the graph, this partial marginal contribution is exact. When all incoming messages arrive at a_i, multiplying them combines disjoint subtrees around a_i. Normalizing gives the exact marginal.

Thus on a tree factor graph,

$$b_i(a_i) = p(a_i) \tag{17}$$

exactly. Since a chain is connected and acyclic, a chain is a tree.

Remark 1. *The reason BP is exact here is the Markov-chain structure of the clean prior over the sequence index plus independent sitewise noising. It is not merely the fact that the forward diffusion process is Markov in diffusion time. Diffusion-time Markovianity and sequence-index Markovianity are different statements.*

5 Forward-backward BP for the posterior chain

For the chain posterior (7), generic BP becomes the forward-backward algorithm.

Define left messages L_i and right messages R_i so that the local likelihood ℓ_i is kept outside the incoming message at site i. Initialize

$$L_1(a_1) = \mu(a_1), \qquad R_n(a_n) = 1. \tag{18}$$

Then for $i = 1, \dots, n-1$,

$$L_{i+1}(a_{i+1}) = \int_{\mathcal{A}} K_{i+1}(a_{i+1} \mid a_i) L_i(a_i) \ell_i(a_i; x_i) \mathrm{d}a_i. \tag{19}$$

For $i = n, n-1, \dots, 2$,

$$R_{i-1}(a_{i-1}) = \int_{\mathcal{A}} K_i(a_i \mid a_{i-1}) R_i(a_i) \ell_i(a_i; x_i) \mathrm{d}a_i. \tag{20}$$

The posterior belief at site i is

$$b_i(a_i) = \frac{L_i(a_i) \ell_i(a_i; x_i) R_i(a_i)}{\int_{\mathcal{A}} L_i(u) \ell_i(u; x_i) R_i(u) \, \mathrm{d}u}. \tag{21}$$

Because the posterior graph is a chain,

$$b_i(a_i) = p_t(a_i \mid x_{1:n}). \tag{22}$$

Therefore the exact posterior mean is

$$m_i(x, t) := \mathbb{E}_t[a_i \mid x_{1:n}] = \int_{\mathcal{A}} a_i b_i(a_i) \, \mathrm{d}a_i. \tag{23}$$

6 The score identity: every algebraic step

This is the key diffusion-specific identity. The noisy marginal is

$$p_t(x_{1:n}) = \int_{\mathcal{A}^n} p_0(a_{1:n}) \prod_{j=1}^{n} \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) \mathrm{d}a_{1:n}. \tag{24}$$

For compact notation define

$$g_j(x_j \mid a_j) := \mathcal{N}(x_j; \alpha_t a_j, \Delta_t). \tag{25}$$

Then

$$p_t(x) = \int p_0(a) \prod_{j=1}^{n} g_j(x_j \mid a_j) \, \mathrm{d}a. \tag{26}$$

We compute the i-th score component:

$$s_i(x,t) = \partial_{x_i} \log p_t(x). \tag{27}$$

Using

$$\partial_{x_i} \log p_t(x) = \frac{\partial_{x_i} p_t(x)}{p_t(x)}, \tag{28}$$

we first differentiate the marginal density. Under the usual regularity/dominated-convergence conditions that hold for the Gaussian likelihoods used here,

$$\partial_{x_i} p_t(x) = \partial_{x_i} \int p_0(a) \prod_{j=1}^{n} g_j(x_j \mid a_j) \, \mathrm{d}a \tag{29}$$

$$= \int p_0(a) \partial_{x_i} \left[\prod_{j=1}^{n} g_j(x_j \mid a_j) \right] \mathrm{d}a. \tag{30}$$

Only the factor $g_i(x_i \mid a_i)$ depends on x_i, so

$$\partial_{x_i} \left[\prod_{j=1}^{n} g_j(x_j \mid a_j) \right] = [\partial_{x_i} g_i(x_i \mid a_i)] \prod_{j \neq i} g_j(x_j \mid a_j). \tag{31}$$

Now compute the derivative of the Gaussian factor explicitly:

$$g_i(x_i \mid a_i) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp\left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right], \tag{32}$$

$$\log g_i(x_i \mid a_i) = -\frac{1}{2} \log(2\pi\Delta_t) - \frac{(x_i - \alpha_t a_i)^2}{2\Delta_t}, \tag{33}$$

$$\partial_{x_i} \log g_i(x_i \mid a_i) = -\frac{1}{2\Delta_t} \partial_{x_i} (x_i - \alpha_t a_i)^2, \tag{34}$$

$$= -\frac{1}{2\Delta_t} 2(x_i - \alpha_t a_i), \tag{35}$$

$$= -\frac{x_i - \alpha_t a_i}{\Delta_t}. \tag{36}$$

Therefore

$$\partial_{x_i} g_i(x_i \mid a_i) = g_i(x_i \mid a_i) \partial_{x_i} \log g_i(x_i \mid a_i) = -\frac{x_i - \alpha_t a_i}{\Delta_t} g_i(x_i \mid a_i). \tag{37}$$

Substitute (37) into (30):

$$\partial_{x_i} p_t(x) = \int p_0(a) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} g_i(x_i \mid a_i) \right] \prod_{j \neq i} g_j(x_j \mid a_j) \mathrm{d}a \tag{38}$$

$$= \int p_0(a) \prod_{j=1}^{n} g_j(x_j \mid a_j) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] \mathrm{d}a. \tag{39}$$

Divide by $p_t(x)$:

$$\partial_{x_i} \log p_t(x) = \frac{1}{p_t(x)} \int p_0(a) \prod_{j=1}^{n} g_j(x_j \mid a_j) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] \mathrm{d}a. \tag{40}$$

Now use Bayes' rule in density form:

$$p_t(a \mid x) = \frac{p_0(a) \prod_{j=1}^{n} g_j(x_j \mid a_j)}{p_t(x)}. \tag{41}$$

Thus

$$\partial_{x_i} \log p_t(x) = \int p_t(a \mid x) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] \mathrm{d}a \tag{42}$$

$$= -\frac{1}{\Delta_t} \int p_t(a \mid x)(x_i - \alpha_t a_i) \, \mathrm{d}a \tag{43}$$

$$= -\frac{1}{\Delta_t} \left[x_i \int p_t(a \mid x) \, \mathrm{d}a - \alpha_t \int a_i p_t(a \mid x) \, \mathrm{d}a \right] \tag{44}$$

$$= -\frac{1}{\Delta_t} \left[x_i - \alpha_t \mathbb{E}_t[a_i \mid x] \right]. \tag{45}$$

Therefore

$$\boxed{s_i(x,t) = \partial_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t m_i(x,t)}{\Delta_t}, \qquad m_i(x,t) = \mathbb{E}_t[a_i \mid x].} \tag{46}$$

Combining (22), (23), and (46) gives the main theorem.

Theorem 1 (Exact score by BP for Markov-chain data under OU noising)**.** *Assume*

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^{n} K_i(a_i \mid a_{i-1})$$

and

$$x_i = \alpha_t a_i + \sqrt{\Delta_t} z_i, \qquad z_i \sim \mathcal{N}(0,1)$$

independently over i. Then $p_t(a_{1:n} \mid x_{1:n})$ is a chain-structured factor graph. Sum-product BP computes the exact posterior marginal

$$b_i(a_i) = p_t(a_i \mid x_{1:n}).$$

If

$$m_i^{\mathrm{BP}}(x,t) = \int a_i b_i(a_i) \, \mathrm{d}a_i,$$

then

$$m_i^{\mathrm{BP}}(x,t) = \mathbb{E}_t[a_i \mid x],$$

and the exact diffusion score is

$$\boxed{s_i(x,t) = -\frac{x_i - \alpha_t m_i^{\mathrm{BP}}(x,t)}{\Delta_t}.} \tag{47}$$

In vector form,

$$\boxed{\nabla_x \log p_t(x) = -\frac{x - \alpha_t m^{\mathrm{BP}}(x,t)}{\Delta_t}.} \tag{48}$$

7 What was special about the Gaussian AR(1) case?

7.1 Gaussian clean prior

In the Gaussian AR(1) case,

$$a_1 \sim \mathcal{N}(0,1), \qquad a_i = \rho a_{i-1} + \sqrt{q}\,\xi_i, \qquad q = 1-\rho^2, \tag{49}$$

with $\xi_i \sim \mathcal{N}(0,1)$. The transition is

$$K_i(a_i \mid a_{i-1}) = \mathcal{N}(a_i; \rho a_{i-1}, q). \tag{50}$$

The covariance matrix of the clean chain is

$$\Sigma_0(i,j) = \rho^{|i-j|}. \tag{51}$$

The noisy vector is also Gaussian:

$$x = \alpha_t a + \sqrt{\Delta_t} z, \qquad a \sim \mathcal{N}(0,\Sigma_0), \qquad z \sim \mathcal{N}(0,I), \tag{52}$$

so

$$x \sim \mathcal{N}(0,\Sigma_t), \qquad \Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I. \tag{53}$$

For any centered Gaussian density

$$p(x) \propto \exp\left(-\frac{1}{2} x^\top \Sigma^{-1} x\right),$$

we have

$$\nabla_x \log p(x) = -\Sigma^{-1} x. \tag{54}$$

Therefore the exact Gaussian score is

$$\boxed{s_{\mathrm{G}}(x,t) = -\Sigma_t^{-1} x.} \tag{55}$$

The exact Gaussian posterior mean is

$$\boxed{m_{\mathrm{G}}(x,t) = \alpha_t \Sigma_0 \Sigma_t^{-1} x.} \tag{56}$$

One can verify directly that (55) and (46) agree:

$$-\frac{x - \alpha_t m_{\mathrm{G}}}{\Delta_t} = -\frac{x - \alpha_t(\alpha_t \Sigma_0 \Sigma_t^{-1} x)}{\Delta_t} \tag{57}$$

$$= -\frac{[I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}] x}{\Delta_t}. \tag{58}$$

Since

$$\Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I, \tag{59}$$

we have

$$\Delta_t I = \Sigma_t - \alpha_t^2 \Sigma_0. \tag{60}$$

Multiplying on the right by Σ_t^{-1},

$$\Delta_t \Sigma_t^{-1} = I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}. \tag{61}$$

Hence

$$-\frac{[I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}]x}{\Delta_t} = -\frac{\Delta_t \Sigma_t^{-1} x}{\Delta_t} \tag{62}$$

$$= -\Sigma_t^{-1} x. \tag{63}$$

7.2 Gaussian message closure

A Gaussian BP message can be written in information form as

$$m(a) \propto \exp\left(-\frac{1}{2}\lambda a^2 + ha\right), \tag{64}$$

where λ is precision and h is information. Its mean and variance are

$$\mu = \frac{h}{\lambda}, \qquad v = \frac{1}{\lambda}. \tag{65}$$

The OU likelihood as a function of a_i is also Gaussian-shaped:

$$\ell_i(a_i; x_i) \propto \exp\left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t}\right] \tag{66}$$

$$\propto \exp\left[-\frac{\alpha_t^2}{2\Delta_t} a_i^2 + \frac{\alpha_t x_i}{\Delta_t} a_i\right]. \tag{67}$$

Thus multiplying a Gaussian message by the likelihood updates information parameters as

$$\widetilde{\lambda}_i = \lambda_i + \frac{\alpha_t^2}{\Delta_t}, \qquad \widetilde{h}_i = h_i + \frac{\alpha_t x_i}{\Delta_t}. \tag{68}$$

The resulting temporary mean and variance are

$$\widetilde{\mu}_i = \frac{\widetilde{h}_i}{\widetilde{\lambda}_i}, \qquad \widetilde{v}_i = \frac{1}{\widetilde{\lambda}_i}. \tag{69}$$

Now apply the Gaussian transition

$$a_{i+1} = \rho a_i + \sqrt{q}\xi_{i+1}. \tag{70}$$

If a_i has temporary distribution $\mathcal{N}(\widetilde{\mu}_i, \widetilde{v}_i)$, then

$$a_{i+1} \sim \mathcal{N}(\rho\widetilde{\mu}_i, q + \rho^2 \widetilde{v}_i). \tag{71}$$

Therefore the outgoing left message remains Gaussian with

$$\lambda_{i+1} = \frac{1}{q + \rho^2 \widetilde{v}_i}, \qquad h_{i+1} = \frac{\rho\widetilde{\mu}_i}{q + \rho^2 \widetilde{v}_i}. \tag{72}$$

This is the finite-dimensional closure. The message update is exact because products and marginalizations of Gaussian densities remain Gaussian.

7.3 The key distinction

The Gaussian AR(1) case combines two facts:

1. BP is exact because the posterior graph is a chain.
2. Messages are finite-dimensional because the Gaussian family is closed under the BP updates.

The first fact survives for any Markov-chain prior. The second does not.

Therefore:

Gaussianity gives closure, not exactness.

8 The continuous non-Gaussian bottleneck

For a general continuous non-Gaussian transition K_i, the update

$$L_{i+1}(a_{i+1}) = \int K_{i+1}(a_{i+1} \mid a_i) L_i(a_i) \ell_i(a_i; x_i) \mathrm{d}a_i \tag{73}$$

maps one function into another function. Thus the message space is infinite-dimensional:

$$L_i, R_i \in L^1_+(\mathbb{R}). \tag{74}$$

This is the real computational issue. Exact BP exists mathematically, but one cannot store arbitrary functions exactly on a computer. We must approximate them.

The simplest hierarchy of approximations is:

Approximation	Message representation	Comment
Grid BP	values on M grid points	reference numerical method
Gaussian projected BP	mean and variance	tests Gaussian closure error
Gaussian mixture BP	mixture weights, means, variances	handles multimodality
Particle BP	weighted samples	Monte Carlo approximation
Basis expansion	coefficients in basis	Hermite/spline/Fourier style
Neural messages	amortized function/embedding	most flexible, but harder to justify

In this project we focus first on the two cleanest algorithms: grid BP and Gaussian projected BP.

9 Approximation error and score error

Let $m_i(x,t) = \mathbb{E}[a_i \mid x]$ be the exact posterior mean and $\widehat{m}_i(x,t)$ an approximation. Define

$$s_i(x,t) = -\frac{x_i - \alpha_t m_i(x,t)}{\Delta_t}, \qquad \widehat{s}_i(x,t) = -\frac{x_i - \alpha_t \widehat{m}_i(x,t)}{\Delta_t}. \tag{75}$$

Then the score error is exactly

$$\widehat{s}_i - s_i = -\frac{x_i - \alpha_t \widehat{m}_i}{\Delta_t} + \frac{x_i - \alpha_t m_i}{\Delta_t} \tag{76}$$

$$= \frac{-x_i + \alpha_t \widehat{m}_i + x_i - \alpha_t m_i}{\Delta_t} \tag{77}$$

$$= \frac{\alpha_t}{\Delta_t}(\widehat{m}_i - m_i). \tag{78}$$

Thus

$$\boxed{\widehat{s}_i(x,t) - s_i(x,t) = \frac{\alpha_t}{\Delta_t}\left[\widehat{m}_i(x,t) - m_i(x,t)\right].} \tag{79}$$

This identity is important because it tells us that all approximation errors enter through posterior-mean error. The OU factor

$$\frac{\alpha_t}{\Delta_t} = \frac{e^{-t}}{1-e^{-2t}} \tag{80}$$

behaves as

$$\frac{e^{-t}}{1-e^{-2t}} \sim \frac{1}{2t} \qquad \text{as } t \downarrow 0. \tag{81}$$

Therefore small-noise times amplify posterior-mean errors strongly.

10 Algorithm 1: reference grid BP

10.1 Grid representation

Choose grid points

$$u_1, \dots, u_M \in [-A, A] \tag{82}$$

and quadrature weights $w_1, \dots, w_M$. In the code, equally spaced points and composite trapezoidal weights are used.

A message $L_i(a)$ is represented by the vector

$$(L_i(u_1), \dots, L_i(u_M)). \tag{83}$$

The transition is represented by the matrix

$$K_{k\ell} = K(u_k \mid u_\ell). \tag{84}$$

The likelihood is represented by

$$\ell_{i,k} = \ell_i(u_k; x_i). \tag{85}$$

10.2 Forward recursion

The continuous update is

$$L_{i+1}(a') = \int K(a' \mid a) L_i(a) \ell_i(a; x_i)\, \mathrm{d}a. \tag{86}$$

The grid approximation at $a' = u_k$ is

$$L_{i+1}(u_k) \approx \sum_{\ell=1}^{M} K(u_k \mid u_\ell) L_i(u_\ell) \ell_i(u_\ell; x_i) w_\ell. \tag{87}$$

In matrix form,

$$L_{i+1} \approx K\left[(L_i \odot \ell_i) \odot w\right], \tag{88}$$

where $\odot$ denotes componentwise multiplication.

10.3 Backward recursion

The continuous update is

$$R_{i-1}(a) = \int K(a' \mid a) R_i(a') \ell_i(a'; x_i) \, \mathrm{d}a'. \tag{89}$$

The grid approximation at $a = u_\ell$ is

$$R_{i-1}(u_\ell) \approx \sum_{k=1}^{M} K(u_k \mid u_\ell) R_i(u_k) \ell_i(u_k; x_i) w_k. \tag{90}$$

In matrix form,

$$R_{i-1} \approx K^\top \left[(R_i \odot \ell_i) \odot w \right]. \tag{91}$$

10.4 Belief, posterior mean, and score

The belief is

$$b_i(u_k) = \frac{L_i(u_k) \ell_i(u_k; x_i) R_i(u_k)}{\sum_{\ell=1}^{M} L_i(u_\ell) \ell_i(u_\ell; x_i) R_i(u_\ell) w_\ell}. \tag{92}$$

The posterior mean is

$$m_i^{\mathrm{grid}} = \sum_{k=1}^{M} u_k b_i(u_k) w_k. \tag{93}$$

The grid score is

$$s_i^{\mathrm{grid}} = -\frac{x_i - \alpha_t m_i^{\mathrm{grid}}}{\Delta_t}. \tag{94}$$

The cost is roughly $O(nM^2)$, because each forward/backward step applies an $M \times M$ transition matrix.

11 Algorithm 2: Gaussian projected BP

Gaussian projected BP forces every message to be Gaussian:

$$L_i(a) \approx \mathcal{N}(a; \mu_i^L, v_i^L), \qquad R_i(a) \approx \mathcal{N}(a; \mu_i^R, v_i^R). \tag{95}$$

For the forward step, first form the exact outgoing message on the grid:

$$\widetilde{L}_{i+1}(u_k) = \sum_{\ell=1}^{M} K(u_k \mid u_\ell) \mathcal{N}(u_\ell; \mu_i^L, v_i^L) \ell_i(u_\ell; x_i) w_\ell. \tag{96}$$

Then project $\widetilde{L}_{i+1}$ back to a Gaussian by moment matching:

$$\mu_{i+1}^L = \sum_{k=1}^{M} u_k \bar{L}_{i+1}(u_k) w_k, \tag{97}$$

$$v_{i+1}^L = \sum_{k=1}^{M} (u_k - \mu_{i+1}^L)^2 \bar{L}_{i+1}(u_k) w_k, \tag{98}$$

where $\bar{L}_{i+1}$ is the normalized version of $\widetilde{L}_{i+1}$.

The backward step is analogous. The approximate belief is

$$b_i^{\mathrm{GMsg}}(a) \propto \mathcal{N}(a; \mu_i^L, v_i^L)\ell_i(a; x_i)\mathcal{N}(a; \mu_i^R, v_i^R). \tag{99}$$

The posterior mean is computed by grid quadrature:

$$m_i^{\mathrm{GMsg}} = \sum_{k=1}^{M} u_k b_i^{\mathrm{GMsg}}(u_k) w_k. \tag{100}$$

The approximate score is

$$s_i^{\mathrm{GMsg}} = -\frac{x_i - \alpha_t m_i^{\mathrm{GMsg}}}{\Delta_t}. \tag{101}$$

In the Gaussian AR(1) case, this projection is exact in principle. In a non-Gaussian chain, it is an approximation. The goal of the experiments is to measure the induced error.

12 Experimental models

12.1 Gaussian AR(1): validation model

The validation model is

$$a_1 \sim \mathcal{N}(0,1), \qquad a_i = \rho a_{i-1} + \sqrt{1-\rho^2}\xi_i, \qquad \xi_i \sim \mathcal{N}(0,1). \tag{102}$$

This gives the exact precision-matrix score (55). Therefore we can directly evaluate the grid discretization error:

$$\mathrm{GridErr}_{\mathrm{G}}(t, M) = \frac{\|s^{\mathrm{G}}_{\mathrm{grid}}(x,t) - s^{\mathrm{G}}_{\mathrm{matrix}}(x,t)\|_2}{\|s^{\mathrm{G}}_{\mathrm{matrix}}(x,t)\|_2}. \tag{103}$$

12.2 Laplace-innovation AR(1): non-Gaussian test model

The non-Gaussian model is

$$a_1 \sim \mathcal{N}(0,1), \qquad a_i = \rho a_{i-1} + \varepsilon_i, \tag{104}$$

where

$$\varepsilon_i \sim \mathrm{Laplace}(0, b). \tag{105}$$

The Laplace density is

$$p(\varepsilon) = \frac{1}{2b}\exp\left(-\frac{|\varepsilon|}{b}\right). \tag{106}$$

A Laplace$(0, b)$ variable has variance $2b^2$. We choose

$$b = \sqrt{\frac{1-\rho^2}{2}} \tag{107}$$

so

$$\mathrm{Var}(\varepsilon_i) = 2b^2 = 1 - \rho^2. \tag{108}$$

Thus if $\mathrm{Var}(a_{i-1}) = 1$, then

$$\begin{aligned} \mathrm{Var}(a_i) &= \mathrm{Var}(\rho a_{i-1} + \varepsilon_i) && (109)\\ &= \rho^2 \mathrm{Var}(a_{i-1}) + \mathrm{Var}(\varepsilon_i) && (110)\\ &= \rho^2 + (1-\rho^2) && (111)\\ &= 1. && (112) \end{aligned}$$

Also, since ε_i is independent of the past,

$$\mathrm{Cov}(a_i, a_{i-k}) = \rho^k \,\mathrm{Var}(a_{i-k}) = \rho^k. \tag{113}$$

So the second-order covariance structure is still AR(1), but the distribution is non-Gaussian. This makes it a clean model for testing what Gaussian message closure loses beyond covariance information.

In this model there is no closed-form precision-matrix score. We therefore use fine-grid BP as a numerical proxy for the true continuous BP score.

13 Error metrics

For posterior means,

$$\mathrm{MSE}_m(t) = \frac{1}{n}\sum_{i=1}^{n}\left(m_i^{\mathrm{GMsg}}(x,t) - m_i^{\mathrm{grid}}(x,t)\right)^2. \tag{114}$$

For score vectors,

$$\mathrm{MSE}_s(t) = \frac{1}{n}\sum_{i=1}^{n}\left(s_i^{\mathrm{GMsg}}(x,t) - s_i^{\mathrm{grid}}(x,t)\right)^2. \tag{115}$$

The relative score error is

$$\mathrm{RelErr}_s(t) = \frac{\|s^{\mathrm{GMsg}}(x,t) - s^{\mathrm{grid}}(x,t)\|_2}{\|s^{\mathrm{grid}}(x,t)\|_2}. \tag{116}$$

The cosine similarity is

$$\mathrm{Cos}(t) = \frac{\langle s^{\mathrm{GMsg}}(x,t), s^{\mathrm{grid}}(x,t)\rangle}{\|s^{\mathrm{GMsg}}(x,t)\|_2 \|s^{\mathrm{grid}}(x,t)\|_2}. \tag{117}$$

The code also verifies the residual of the identity

$$s^{\mathrm{GMsg}} - s^{\mathrm{grid}} = \frac{\alpha_t}{\Delta_t}(m^{\mathrm{GMsg}} - m^{\mathrm{grid}}), \tag{118}$$

which should be at numerical precision.

14 Numerical results from the report run

The report run uses:

Parameter	Value
Chain length	$n = 50$
Correlation	$\rho = 0.85$
Grid interval	$[-8, 8]$
Validation grid sizes	$101, 201, 401$
Laplace convergence grids	201 vs 401
Reference grid size	401
Trials per time	12
Diffusion times	$0.08, 0.12, 0.18, 0.27, 0.40, 0.60, 0.90, 1.30, 1.80, 2.40$

14.1 Gaussian validation of grid BP

Figure 1 compares grid BP with the exact Gaussian precision-matrix score. This validates the grid procedure in a setting where the true score is known analytically.

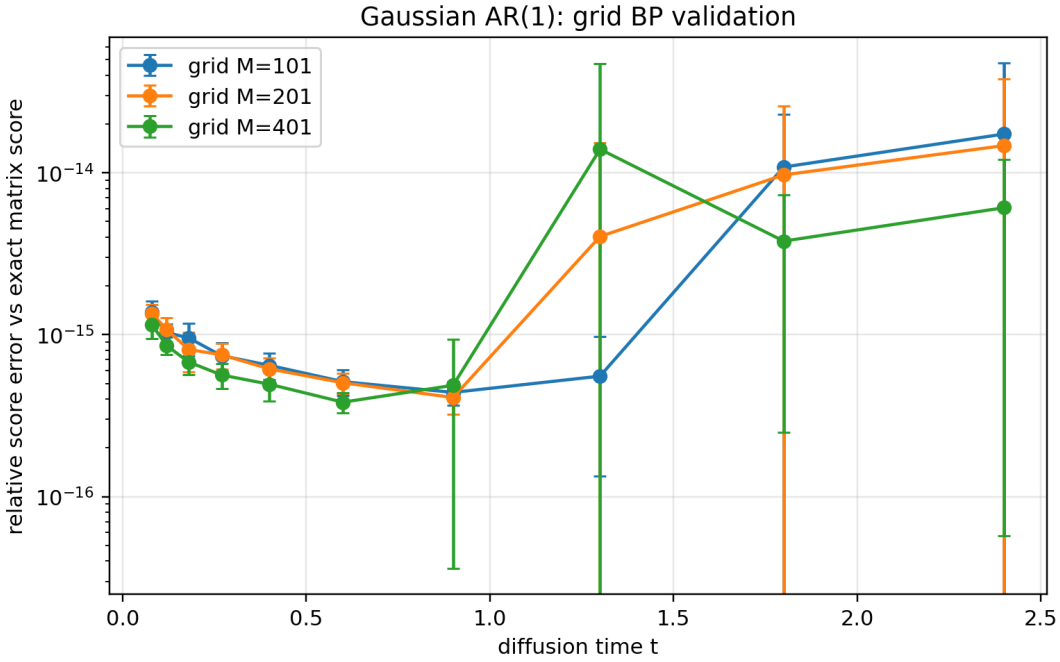

Figure 1: Gaussian AR(1) validation: relative error between grid BP score and exact precision-matrix score. The error is extremely small for the chosen interval and grid sizes, confirming that grid BP is a reliable numerical reference in this controlled case.

Figure 2 shows the Gaussian projected BP sanity check in the Gaussian case. In principle, Gaussian projection is exact for Gaussian transitions and Gaussian likelihoods; remaining deviations are numerical effects from finite grid support and quadrature.

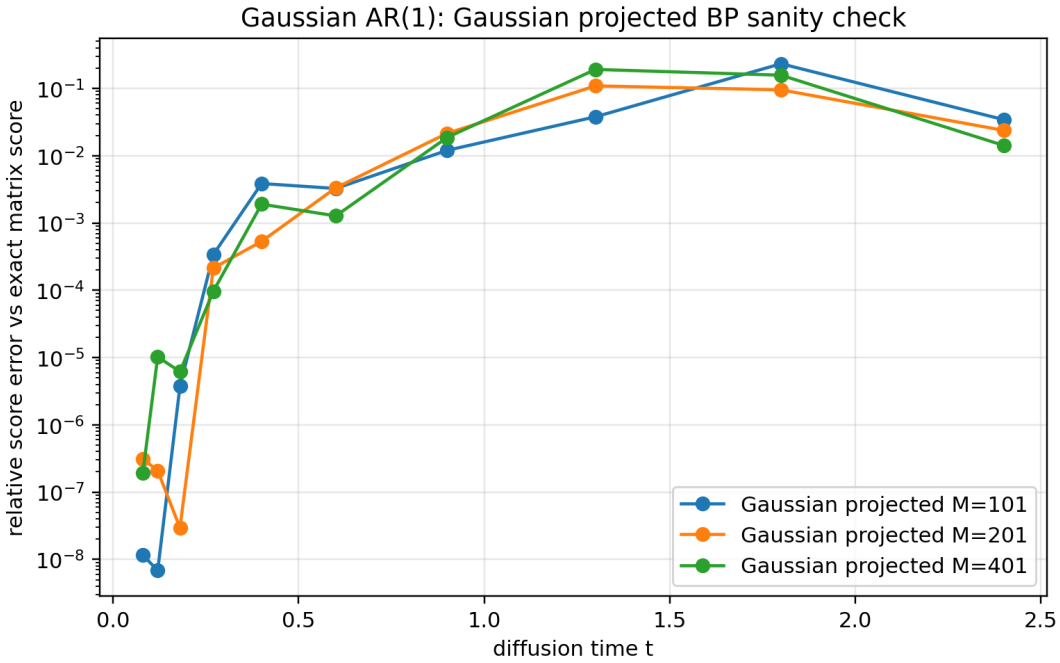

Figure 2: Gaussian AR(1) sanity check for Gaussian projected BP against the exact matrix score. The projection is structurally correct in the Gaussian case; numerical deviations are due to finite-grid implementation details.

14.2 Laplace grid convergence

In the Laplace case there is no precision-matrix score. Figure 3 compares grid BP with $M = 201$ and $M = 401$. This estimates the grid discretization error of the reference method.

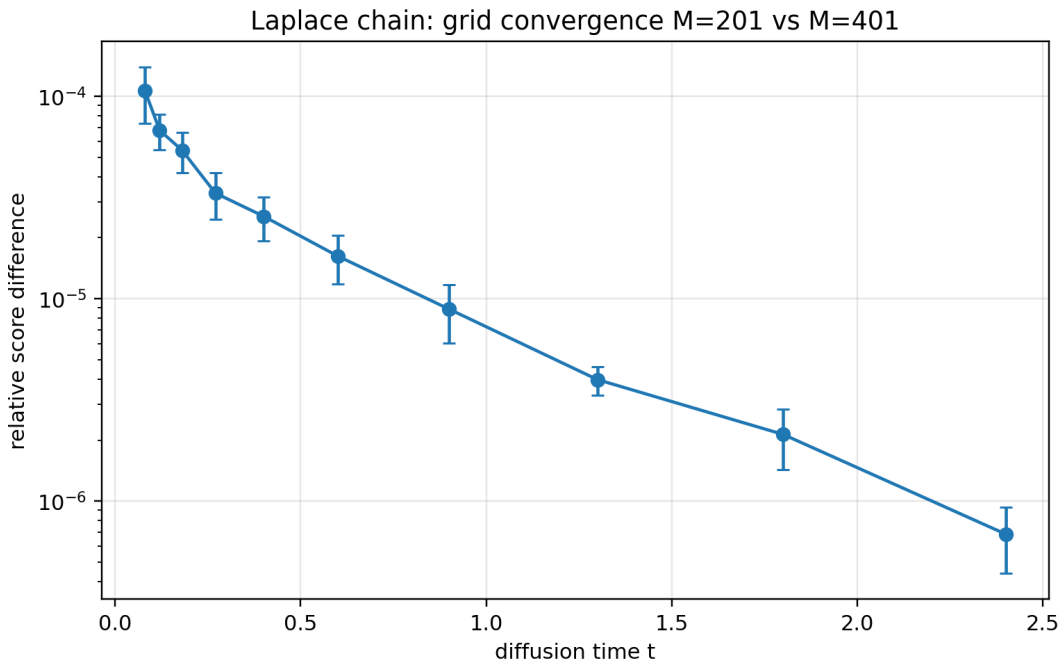

Figure 3: Laplace chain grid convergence: relative score difference between coarse grid $M = 201$ and fine grid $M = 401$. This is our proxy for grid discretization error in the non-Gaussian case.

Representative values from the Laplace grid-convergence CSV are:

t	Relative score difference	Posterior mean MSE	Score cosine
0.08	1.07×10^{-4}	9.54×10^{-10}	0.999999994
0.40	2.55×10^{-5}	5.77×10^{-10}	0.9999999997
1.30	3.98×10^{-6}	2.02×10^{-10}	0.99999999999
2.40	6.86×10^{-7}	5.91×10^{-11}	0.9999999999998

The key interpretation is that the grid-discretization error is much smaller than the Gaussian-message approximation error measured below, at least for the report settings.

14.3 Gaussian-message approximation error in the Laplace chain

Figure 4 shows posterior-mean MSE. Figure 5 shows relative score error. Figure 6 shows score cosine similarity.

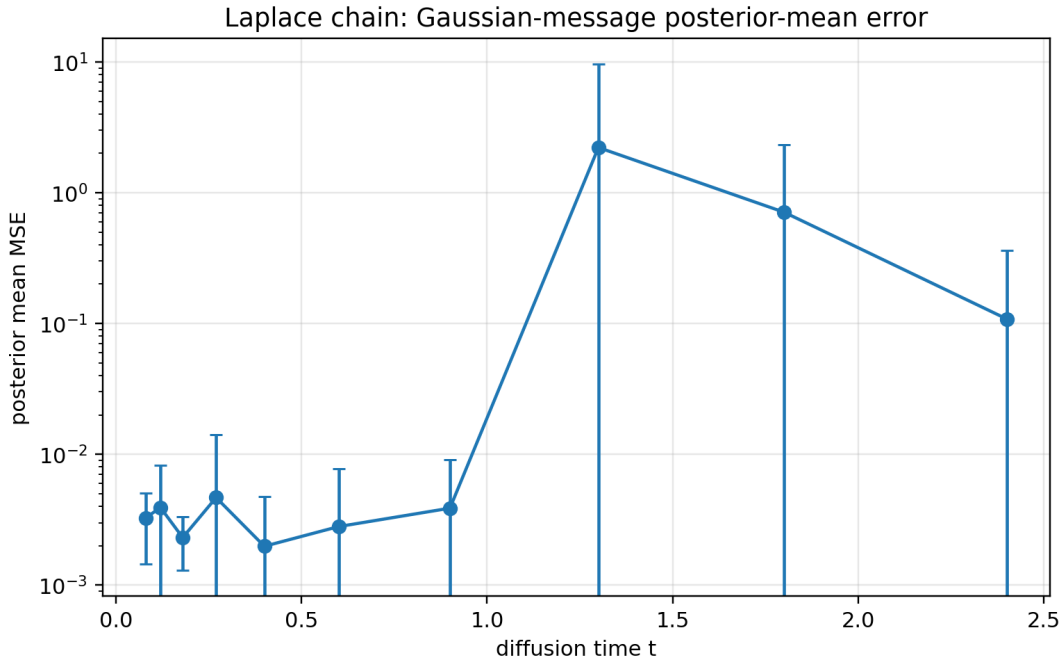

Figure 4: Laplace chain: posterior-mean MSE between Gaussian projected BP and reference grid BP.

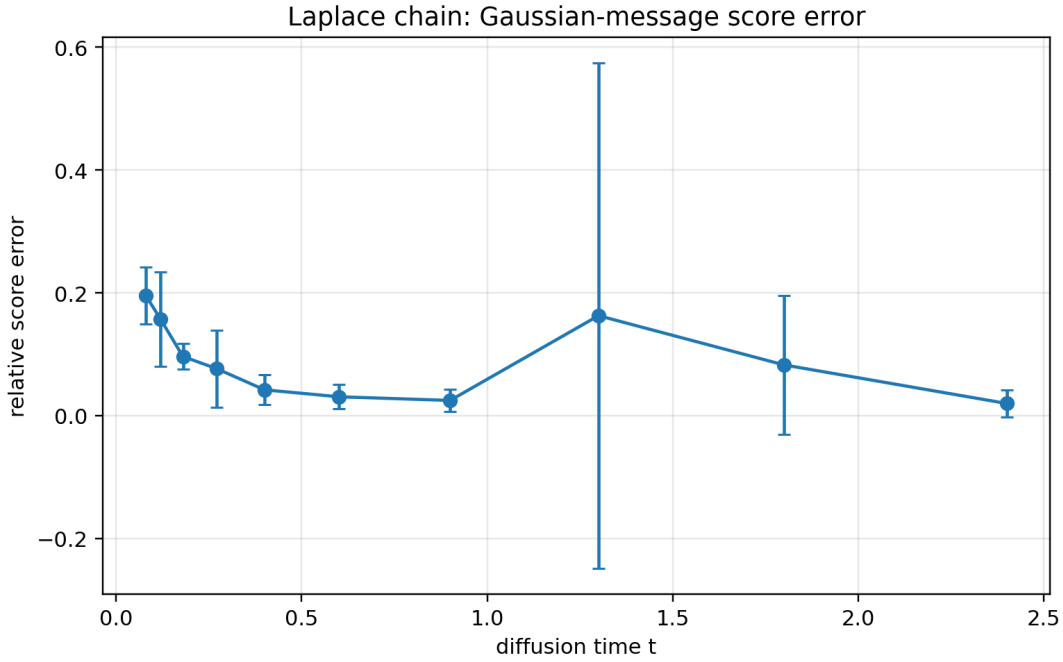

Figure 5: Laplace chain: relative score error of Gaussian projected BP compared with reference grid BP. Errors are largest at small noise, where the amplification factor α_t/Δ_t is large. Some large-time variability appears because the posterior becomes weakly informative and the Gaussian projection can occasionally behave poorly.

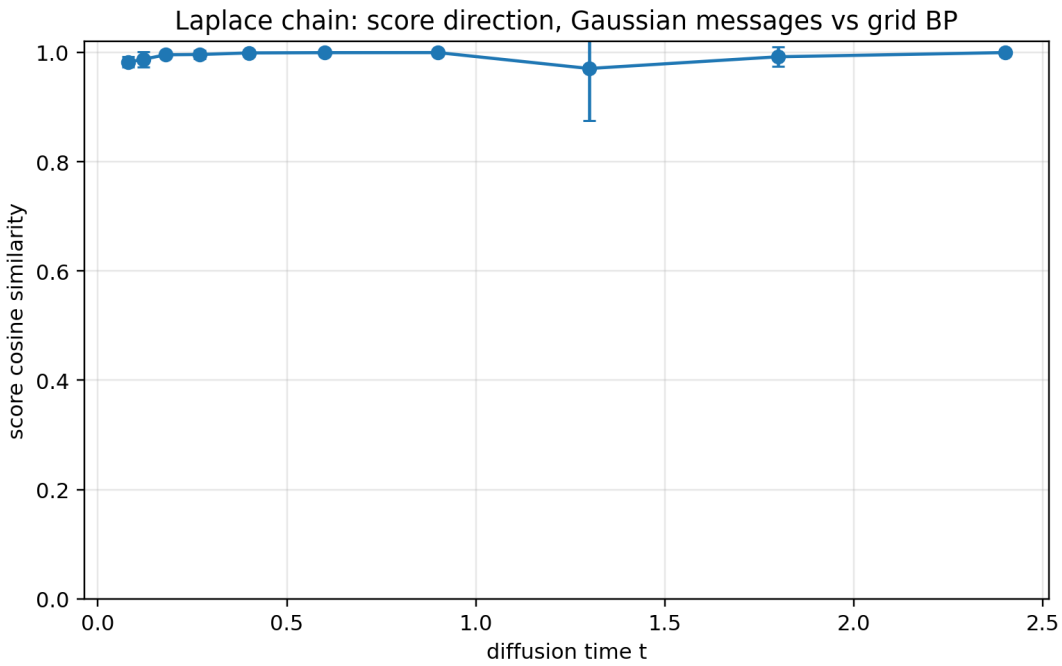

Figure 6: Laplace chain: cosine similarity between Gaussian-message score and reference grid-BP score. Even when the norm error is non-negligible, the direction can remain close for many diffusion times.

Representative values for Gaussian-message approximation error are:

t	Posterior mean MSE	Relative score error	Score cosine
0.08	3.26×10^{-3}	0.195	0.982
0.12	3.92×10^{-3}	0.157	0.987
0.40	1.98×10^{-3}	0.0419	0.999
0.90	3.87×10^{-3}	0.0246	0.9996
1.80	7.10×10^{-1}	0.0825	0.992
2.40	1.08×10^{-1}	0.0197	0.9996

These are preliminary Monte Carlo estimates. The script is designed to increase `-n-trials`, `-ref-grid-size`, or the grid interval for heavier runs.

14.4 Example beliefs

Figure 7 compares a posterior belief at one site. This is the most concrete way to see what Gaussian projection does: it replaces the reference grid posterior shape by a Gaussian-message-induced approximation.

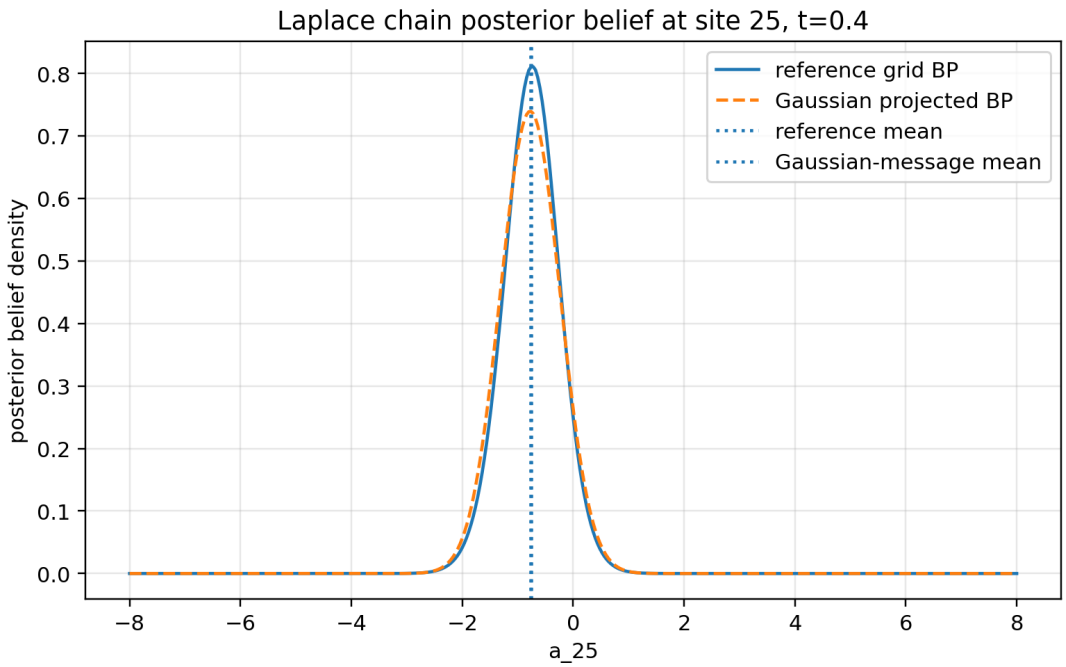

Figure 7: Example posterior marginal in the Laplace chain. The reference grid BP belief is compared with the Gaussian projected BP belief. The vertical lines show the corresponding posterior means.

Figure 8 shows an example Gaussian validation score component comparison.

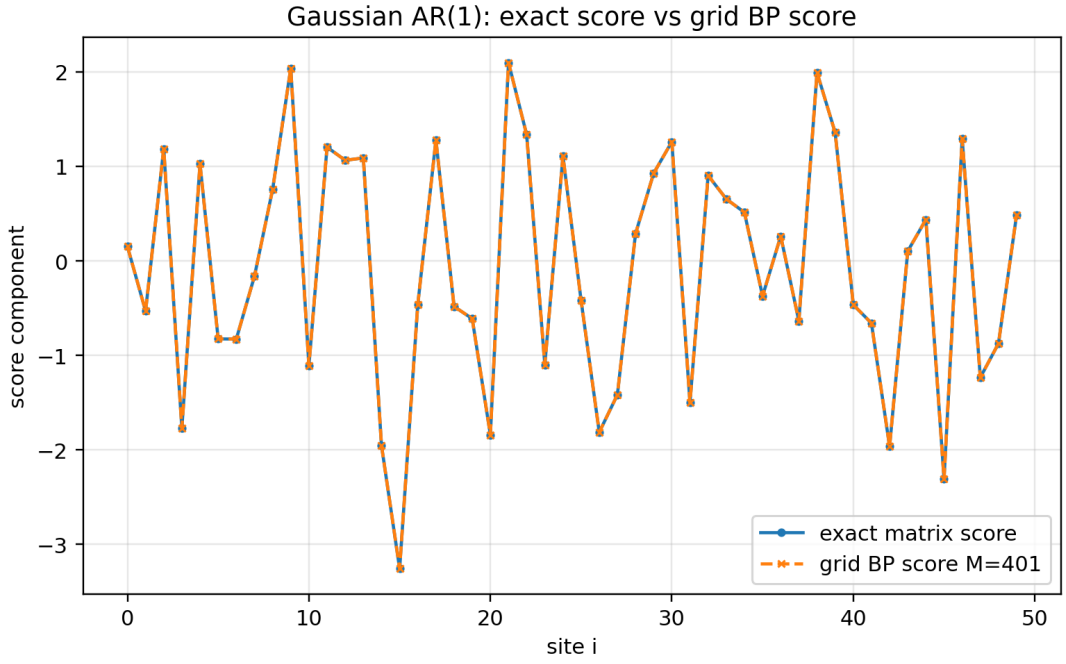

Figure 8: Gaussian AR(1) example: exact precision-matrix score versus grid BP score.

15 Code explanation

The Python script is

`code/bp_general_markov_diffusion_experiments.py`.

It is organized in layers matching the mathematics.

15.1 Configuration

The dataclass `ExperimentConfig` stores all experimental parameters:

```
@dataclass
class ExperimentConfig:
    n: int = 50
    rho: float = 0.85
    grid_min: float = -8.0
    grid_max: float = 8.0
    validation_grid_sizes: tuple[int, ...] = (101, 201, 401)
    convergence_coarse_grid_size: int = 201
    ref_grid_size: int = 401
    n_trials: int = 6
    seed: int = 11
    output_dir: str = "bp_markov_diffusion_outputs"
    t_values: tuple[float, ...] = (...)
```

This makes it easy to rerun heavier experiments by changing `n_trials`, `ref_grid_size`, or the grid interval.

15.2 Densities and numerical normalization

The functions `normal_pdf` and `laplace_pdf` implement the Gaussian and Laplace densities. The function `trapezoid_weights` builds quadrature weights. The function `normalize_density` normalizes grid values as a density:

```
def normalize_density(values: np.ndarray, weights: np.ndarray) -> np.ndarray:
    values = np.maximum(np.asarray(values, dtype=float), 0.0)
    mass = float(np.sum(values * weights))
    if not np.isfinite(mass) or mass <= 0.0:
        raise FloatingPointError("Cannot normalize density.")
    return values / mass
```

Normalization after each message update is numerically important. BP messages are only defined up to multiplicative constants, so normalization does not change beliefs, but it prevents underflow/overflow over long chains.

15.3 Transition matrices

The transition matrix is stored as

$$K[\text{out}, \text{in}] = p(a_{\text{out}} \mid a_{\text{in}}). \tag{119}$$

This convention makes the forward update a matrix-vector product:

$$L_{i+1} = K\left[(L_i \odot \ell_i) \odot w\right]. \tag{120}$$

The code is:

```
def build_transition_matrix(grid, rho, prior):
    q = 1.0 - rho ** 2
    out = grid[:, None]
    inn = grid[None, :]
    if prior == "gaussian":
        return normal_pdf(out, rho * inn, q)
    if prior == "laplace":
        return laplace_pdf(out, rho * inn, np.sqrt(q / 2.0))
```

15.4 Grid BP implementation

The core of `grid_bp` mirrors equations (87) and (90):

```
L[0] = normalize_density(normal_pdf(grid, 0.0, 1.0), weights)
for i in range(n - 1):
    incoming = L[i] * ell[i] * weights
    L[i + 1] = K @ incoming
    L[i + 1] = normalize_density(L[i + 1], weights)

R[-1] = np.ones(m)
for i in range(n - 1, 0, -1):
    incoming = R[i] * ell[i] * weights
    R[i - 1] = K.T @ incoming
    R[i - 1] = normalize_density(R[i - 1], weights)
```

After the two passes, beliefs and means are computed as:

```
b = normalize_density(L[i] * ell[i] * R[i], weights)
means[i] = np.sum(grid * b * weights)
```

This is exactly the discrete quadrature version of the exact BP belief formula.

15.5 Gaussian projected BP implementation

The function `gaussian_projected_bp` stores each message by a mean and variance. For each step, it evaluates the current Gaussian message on the grid, applies the exact grid update, then moment-matches the outgoing density:

```
L_values = gaussian_message_values(grid, L_mean[i], L_var[i])
incoming = L_values * ell[i] * weights
outgoing = K @ incoming
L_mean[i + 1], L_var[i + 1] = density_moments(outgoing, grid, weights)
```

A flat terminal right message is represented by infinite variance:

```
R_mean[-1] = 0.0
R_var[-1] = np.inf
```

The helper `gaussian_message_values` returns a vector of ones when the variance is infinite.

15.6 Exact Gaussian score functions

For validation, the code builds

$$\Sigma_0(i,j) = \rho^{|i-j|}, \qquad \Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I, \tag{121}$$

and computes

$$s = -\Sigma_t^{-1} x \tag{122}$$

using `np.linalg.solve`:

```
def gaussian_precision_score(x, rho, alpha, delta):
    sigma0 = ar1_covariance(len(x), rho)
    sigmat = alpha ** 2 * sigma0 + delta * np.eye(len(x))
    return -np.linalg.solve(sigmat, x)
```

Using `solve` is numerically preferable to explicitly computing Σ_t^{-1}.

15.7 Experiment runners

The script has three experiment runners:

1. `run_gaussian_grid_validation`: compares grid BP and Gaussian projected BP against the exact matrix score.
2. `run_laplace_grid_convergence`: compares coarse-grid and fine-grid BP in the Laplace chain.
3. `run_laplace_gaussian_message_error`: compares Gaussian projected BP against reference grid BP in the Laplace chain.

The final function `run_all` executes all experiments, writes CSV files, and generates plots.

15.8 How to run

From the root folder of the delivered archive:

```
python code/bp_general_markov_diffusion_experiments.py --mode quick
```

for a fast smoke test, or

```
python code/bp_general_markov_diffusion_experiments.py --mode report
```

for the report-quality run. To increase experiment weight:

```
python code/bp_general_markov_diffusion_experiments.py \
    --mode report \
    --n-trials 50 \
    --ref-grid-size 801 \
    --output-dir outputs_heavy
```

The cost grows approximately as $O(nM^2)$, so increasing M is much more expensive than increasing the number of trials.

16 Interpretation and next steps

The clean conclusion is:

BP gives the exact score for Markov-chain priors under OU noising.

The hard part is not the score formula. The hard part is continuous message representation.

The Gaussian approximation should therefore be studied as a message-representation approximation. In the Laplace-innovation experiment, the grid error is tiny compared with the Gaussian-message error, so the observed discrepancy primarily reflects Gaussian closure error rather than grid error.

Natural next steps are:

1. Increase trials and grid size to reduce Monte Carlo variability.
2. Explore different ρ, because stronger correlations make messages more informative and potentially more non-Gaussian.
3. Compare Laplace, Student, Gaussian-mixture, and bounded-support innovations.
4. Add Gaussian-mixture message approximations to see how many mixture components are needed to beat single-Gaussian closure.
5. Study whether neural message approximators help only when they preserve the local BP update structure.

The proposed thesis direction is not that neural networks are unnecessary in general. In real diffusion models, the clean prior p_0 is unknown and must be learned from data. The precise claim is more structural:

> Score learning can be reformulated as posterior denoising. For Markov-chain priors, posterior denoising is exact BP. When exact continuous BP is not computationally closed, one must approximate the messages. Gaussian messages are the first approximation to test, and their error can be measured directly through the posterior-mean/score identity.

A Files in the delivered folder

The delivered folder contains:

File	Purpose
`report/bp_markov_diffusion_gaussian_approx.pdf`	this report as PDF
`report/bp_markov_diffusion_gaussian_approx.tex`	LaTeX source
`code/bp_general_markov_diffusion_experiments.py`	full experiment script
`code/bp_general_markov_diffusion_experiments.ipynb`	executable/exploratory notebook
`outputs/*.csv`	numerical experiment tables
`outputs/*.png`	plots used in the report

B Source references

References

[1] G. Mantovani, *Belief Propagation for the Gaussian AR(1) Diffusion Model: A complete derivation and numerical comparison with the precision-matrix score*, internal project note, 2026.

[2] M. Mezard and A. Montanari, *Information, Physics, and Computation*, Oxford University Press, 2009. Relevant background: factor graphs and belief propagation on trees.

[3] F. Krzakala and L. Zdeborova, *Statistical Physics Methods in Optimization and Machine Learning*, lecture notes, 2024. Relevant background: BP equations, sparse graphs, and the Bethe perspective.

[4] G. Biroli and M. Mezard, *Generative diffusion in very large dimensions*, arXiv:2306.03518, 2023.

[5] G. Biroli, T. Bonnaire, V. de Bortoli, and M. Mezard, *Dynamical Regimes of Diffusion Models*, arXiv:2402.18491, 2024.

[6] S. Mei, *U-Nets as Belief Propagation: Efficient Classification, Denoising, and Diffusion in Generative Hierarchical Models*, arXiv:2404.18444, 2024.

[7] G. Genovese and A. Piana, *Derivation of the AMP Equations from Belief Propagation for the ℓ_2 Minimisation Problem*, arXiv:2602.15191, 2026.